

RESUMO DO MÓDULO

Middleware na Prática - Application-level Middleware

No Express.js, o uso de Application-level Middleware é uma prática essencial para definir funcionalidades que serão executadas em todas as requisições de um aplicativo. Esses middlewares são aplicados diretamente ao objeto app e são usados para processar as requisições antes que elas cheguem às rotas específicas.

O que é um Application-level Middleware?

Um Application-level Middleware é uma função que recebe três argumentos:

1. req (request) - Objeto de requisição que contém informações sobre a solicitação feita pelo cliente.
2. res (response) - Objeto de resposta que permite enviar uma resposta ao cliente.
3. next - Função que permite passar a execução para o próximo middleware na cadeia.

Esses middlewares são definidos usando o método `app.use()` do Express. Eles podem ser utilizados para adicionar lógica comum a todas as rotas, como autenticação, logging, validação, entre outras.

Implementação de um Application-level Middleware

Vamos ver um exemplo prático de como criar um Application-level Middleware que verifica se o usuário está logado antes de permitir o acesso às rotas.

javascript

```
const express = require('express');
```

```
const app = express();

// Definindo um Application-level Middleware para verificar se o usuário está logado
app.use((req, res, next) => {

  const isLoggedIn = true; // Supondo que o usuário está logado

  if (isLoggedIn) {

    next(); // Passa a requisição para o próximo middleware ou rota

  } else {

    res.status(401).send('Você precisa estar logado para acessar esta página!');

  }

});

// Definindo a rota raiz
app.get('/', (req, res) => {

  res.send('Olá, mundo!');

});

// Inicializando o servidor
app.listen(3000, () => {

  console.log('Servidor iniciado em http://localhost:3000');

});
```

Explicação do Código

- O `app.use()` é usado para definir um middleware que será executado em todas as requisições.
- A função de middleware verifica se o usuário está logado (aqui usamos uma variável fictícia `isLoggedIn` para simular a autenticação).

- Se `isLogged` for `true`, o middleware chama `next()` para passar a requisição para a próxima função (pode ser outro middleware ou a própria rota).
- Se `isLogged` for `false`, o middleware envia uma resposta de status 401 informando que o usuário não está autenticado.

Quando você acessa a rota `/`, a função de middleware é executada primeiro. Como `isLogged` é `true`, a requisição é passada para a rota `get('/')`, que responde com a mensagem "Olá, mundo!".

Por que usar Application-level Middleware?

Application-level Middleware é útil para implementar lógica que se aplica a várias rotas do aplicativo, como:

- Autenticação e Autorização: Verificar se o usuário está autenticado antes de acessar rotas protegidas.
- Registro de Logs: Armazenar informações sobre as requisições, como IP do cliente, método HTTP, etc.
- Validação de Dados: Validar as informações recebidas nas requisições antes de passar para o próximo middleware.

Esses middlewares tornam o código mais modular e organizado, facilitando a manutenção e a legibilidade do projeto.

Observação sobre a Utilização de `next()`

O método `next()` é essencial para que o Express saiba que o middleware terminou de processar a requisição e que ela pode ser passada para o próximo middleware ou rota. Se `next()` não for chamado, a requisição ficará pendente e o cliente não receberá uma resposta.

Por exemplo, no código a seguir, se omitirmos a chamada de `next()`, a requisição nunca será passada para a rota `/`:

javascript

```
app.use((req, res, next) => {  
  console.log('Este middleware foi executado.');
```

`// next();` <- Se omitido, a rota abaixo nunca será executada.

```
});
```

Por isso, é fundamental lembrar de chamar `next()` quando a intenção é que a requisição continue fluindo pela cadeia de middlewares e chegue até a rota correta.

Conclusão

O Application-level Middleware é uma peça fundamental no desenvolvimento com Express.js. Ele permite adicionar funcionalidades a todas as rotas de uma forma centralizada, melhorando a estrutura e organização do código. Entender como criar e usar esses middlewares corretamente é um passo importante para construir aplicações robustas e escaláveis com Express.
